

Prelab No. 4

Getting Started with the Raspberry Pi Pico, MicroPython, and Wokwi

Computer Structure Class — Spring 2025
Sharif University of Technology — By Dr. Saeed Bagheri Shouraki

I. Objective

This prelab opens the second half of the lab series, in which you will program the Raspberry Pi Pico using MicroPython. The goal is to get you set up and comfortable before you walk into the lab: you should understand what the RP2040 microcontroller and MicroPython are, know how to build and run a Pico project inside the Wokwi online simulator, interact with your program over a serial terminal, and grasp the GPIO concepts (driving the onboard LED, reading a push button, and debouncing) that the Lab 7 tasks build on. By the end of this prelab you should be able to write, run, and share a small MicroPython program entirely in your browser.

II. Why Use a Simulator?

For this part of the course we run the labs inside Wokwi, a browser-based electronics simulator, rather than on a physical board. There are several practical reasons for this:

- No hardware required. Every student can complete the work from any computer with a browser — there is nothing to buy, borrow, or carry, and no driver to install.
- Nothing to damage. You cannot burn out a pin, short a supply rail, or fry the chip. This lets you experiment freely, which is exactly what learning a new microcontroller calls for.
- Faster iteration and easier help. There is no flashing cable and no waiting; you press Run and see results immediately. Sharing a project is just sharing a link, so a TA can open your exact circuit and code to help you.
- Reproducible grading. Everyone targets the same virtual board, so behavior does not depend on a particular cable, USB port, or board revision.

Note

A simulator is a learning and prototyping tool, not a perfect replica of hardware. A few features (true analog waveforms, multiple independent boards, and some sensor values) behave differently or are unavailable in simulation. Where a task depends on such a feature, the lab instructions will tell you how to adapt it.

III. Concepts and Ideas

III.1. The Raspberry Pi Pico and the RP2040

The Raspberry Pi Pico is a microcontroller development board built around the **RP2040** chip, which contains a dual-core ARM Cortex-M0+ processor, internal RAM, and a flexible Programmable I/O (PIO) block. For this lab the most important details are that general-

purpose pins are named GP0 through GP22, and that the board has an onboard LED wired to GP25. Three pins (GP26, GP27, GP28) double as analog inputs.

III.2. What is MicroPython?

MicroPython is a compact implementation of Python 3 designed to run directly on microcontrollers. Instead of compiling and flashing C, you write ordinary Python that the on-chip interpreter executes. Hardware is accessed through the built-in machine module — for example `Pin` for GPIO, and later `PWM` and `Timer`. A minimal example that turns the onboard LED on looks like this:

```
from machine import Pin
import time

led = Pin(25, Pin.OUT)    # onboard LED
led.value(1)              # turn on (led.on() does the same)
time.sleep(1)             # wait one second
led.value(0)              # turn off
```

III.3. The Wokwi Simulator

Wokwi (<https://wokwi.com>) is an online simulator for the Raspberry Pi Pico and many other boards. You build a circuit visually, write your code in the editor, press Run, and watch it execute — LEDs light up, the serial monitor prints, and inputs respond. For this course you will use the MicroPython template for the Pi Pico. The free plan is sufficient for all Lab 7 tasks.

III.4. Interacting Through the Serial Terminal

Your program talks to you through a **serial terminal**. On real hardware this is a tool such as Thonny; in Wokwi it is the built-in **Serial Monitor** panel. Output works through `print(...)`, and input works through `input(...)`, which pauses the program until you type a line and press Enter. Several Lab 7 tasks ask the user to type values (for example, on/off times), so you read them like this:

```
on_ms = int(input("On time (ms): "))
off_ms = int(input("Off time (ms): "))
```

III.5. GPIO: Outputs, Inputs, and Pull Resistors

A GPIO pin can be configured as an output (your code sets it high or low, e.g. to drive an LED) or as an input (your code reads whether it is high or low, e.g. to sense a button). A bare input pin that is connected to nothing is *floating* and reads unpredictable values. To avoid this, the RP2040 provides internal **pull-up** and **pull-down** resistors that gently tie the pin to a known level. A button in `Pin.PULL_UP` mode reads 1 when released and 0 when pressed (the button connects the pin to ground):

```
btn = Pin(15, Pin.IN, Pin.PULL_UP)
pressed = (btn.value() == 0)    # active-low with a pull-up
```

III.6. Switch Bouncing and Debouncing

When a mechanical button is pressed or released, its metal contacts do not settle cleanly. They bounce — opening and closing rapidly for a few microseconds up to tens of milliseconds — before reaching a stable state. A program that simply counts every transition will register one press as many. The standard cure is debouncing: after detecting an edge, wait a short fixed time (typically 10–50 ms) and ignore any further changes during that window, so the contacts can settle. You will use this idea directly in Lab 4.

IV. Getting Started with Wokwi (Setup)

Complete these steps before the lab so you arrive ready to work:

1. Create a free account at <https://wokwi.com>.
2. Start a new MicroPython project for the Pico from <https://wokwi.com/projects/new/micropython-pi-pico>. You will see a Pi Pico, a `main.py` editor, and a `diagram.json` tab.
3. Add components with the blue “+” button in the simulation panel (for Lab 7 you will add push buttons). Drag a part to position it, then click one pin and the target pin to wire them.
4. Write your code in `main.py`, press the green Run button, and use the Serial Monitor panel at the bottom to read output and type input.
5. Save the project and use Share to get a link. You will submit Wokwi links for the coding parts.

Note

The onboard LED is on GP25 and is already present on the simulated board — you do not need to wire it. Avoid using GP26, GP27, GP28 for digital I/O in these labs, and never rely on the 5 V rail.

V. Resources

- Wokwi simulator: <https://wokwi.com>
- New MicroPython Pico project: <https://wokwi.com/projects/new/micropython-pi-pico>
- MicroPython quick reference for the RP2: <https://docs.micropython.org/en/latest/rp2/quickref.html>
- Raspberry Pi Pico Python SDK / getting started: <https://www.raspberrypi.com/documentation/microcontrollers/micropython.html>
- If you also have a physical Pico, you may use [Thonny](#) with the firmware `RPI_PICO-20250415-v1.25.0.uf2` from CW. This is optional; Wokwi is the supported path for this lab.

VI. Tasks

Answer the written questions in your own words and complete the small coding task in Wokwi. Submit this prelab **before** your lab session. For the coding part, include the shared Wokwi link.

VI.1. GPIO and Pull Resistors (25 pts)

- Explain the difference between configuring a GPIO pin as an input and as an output, and give one example use of each on the Pico.
- What is a floating input, and why is it a problem? Explain how an internal pull-up or pull-down resistor solves it.
- For a push button wired between a GPIO pin and ground and configured with `Pin.PULL_UP`, what value does `btn.value()` return when the button is pressed, and when it is released? Explain why.

VI.2. Serial Interaction (20 pts)

- Explain the roles of `print()` and `input()` and how they relate to the serial terminal.
- Write a short snippet that reads two integers (an on-time and an off-time, in milliseconds) typed by the user on a single run, and prints them back. (This is the input half of the Lab 4 blink task.)

VI.3. Debouncing (25 pts)

- Describe, in your own words, what causes a mechanical button to bounce and roughly how long bounce can last.
- Explain how a simple delay-based debounce works and why a delay in the 10–50 ms range is a reasonable choice. What goes wrong if you make the delay far too long, or far too short?
- Sketch (as pseudocode or a few sentences) how you would count clean button presses while ignoring bounce, without using interrupts.

VI.4. First Wokwi Program — Adjustable Blink (30 pts)

In Wokwi, create a MicroPython Pico project that blinks the onboard LED (GP25). The program should ask the user, over the serial terminal, for an on-time and an off-time in milliseconds, then blink the LED with those timings continuously. Run it, confirm the LED blinks in the simulation, and submit the shared Wokwi link together with your `main.py` code. (This is a warm-up for Lab 4 Task IV.1; bring your working project to the lab.)

Note

Do not use hardware interrupts for any task in this series — use polling and delays. Tasks that rely on interrupts will not be accepted.